

CSCI 112

Programming with C

LECTURE: 7

J. L. Pach

OUTLINE:

- printf format
  - type
  - width
  - precision
  - flags
  - modifier

PRINTF

& scanf format specification

```
printf(char format[],  arg1,  arg2 ,...)
scanf (char format[], *arg1, *arg2 ,...)
```

Format = %[flags][width][.precision][modifier]<type>

- Optional:
  - `Flags` - modifiers that alter the formatting or scanning behavior.
  - `Width` - specifies the minimum width of the output field
    or the maximum number of characters to be scanned.
  - `Precision` - controls the precision of floating-point numbers
    or the maximum number of characters to be scanned for strings.
  - `Modifier` - indicates the data type size (e.g., long, short).
- Required:
  - `Type` - specifies the data type of the variable to be formatted or scanned.

<type>

Type & Specifier		Origin	Argument type		Description	
			printf	**scanf**	**printf**	**scanf**
integer	d	decimal	int	int *	signed decimal notation	
	u	unsigned decimal	int	unsigned int *	unsigned decimal notation	
	c	character	int	char *	one unsigned character	characters are placed within the indicated memory if the specified width is greater than 1; Without '\ 0'
	i	integer	int	int *	signed decimal notation (also accepts octal and hexadecimal)	
	o	octal	int	int *	octal notation (with leading 0)	
	x, X	hex	int	int *	hexadecimal notation (with or without leading 0x or 0X)	
string	s	string	char *	char *	characters from the string are printed until a'\ 0'!	string **of non-white space**; at the end will be added '\ 0'
	[…]	specific string	N/A	char *	N/A	Matches the longest non-empty string input characters from the set between brackets; A '\ 0' is added.
	[^…]	negated specific string	N/A	char *	N/A	as above, but only excluding the characters
floating-point number	f	float	float	float *	single precision floating-point number notation	
	lf	long float (double)	double	double *	double precision floating-point number notation	
	e, E	engineer notation	double	float * double *	scientific notation (single or double precision floatingpoint number)	
	g, G	general floating -point representation	float double	float * double *	scientific notation (single or double precision floating -point number) or single or double precision floating -point number	
pointer	p	pointer	(any) void *	N/A	integer value of the pointer	N/A
special	n	number of input characters	N/A	int *	N/A	Writes into the argument the number of characters read so far by this call
	%	literal %	%	N/A	Prints a literal percent sign (%)	N/A

BASIC TYPE

Type & Specifier		Origin	Argument type		Description	
			printf	**scanf**	**printf**	**scanf**
integer	d	decimal	int	int *	signed decimal notation	
	u	unsigned decimal	int	unsigned int *	unsigned decimal notation	
	c	character	int	char *	one unsigned character	
string	s	string	char *	char *	characters from the string are printed until a '\ 0'!	string **of non-white space**; at the end will be added '\ 0'!
floating -point number	f	float	float	float *	single precision floating-point number notation	
	lf	long float (double)	double	double *	double precision floating-point number notation	

Basic types

```
unsigned int x = 65;  int y = -6; char letter ='a'; char * text = "Some Text";
float fRealNumber = 3.0f; double dRealNumber = 3.4e40;
```



Type & Specifier		Origin	Argument type	Example	Result
integer	d	decimal	int	`printf ("%d\n", x);` `printf ("%d\n", y);`	65 -6
	u	unsigned decimal	int	`printf ("%u\n", x);` `printf ("%u\n", y);`	65 4294967290

	c	character	int	`printf ("%c\n", letter);`
				`printf ("%c\n", x);`
				`printf ("%d\n", letter);`
					a
					A
					97

string	s	string	char *	`printf ("%s\n", text);`	Some Text
floating-point number	f	float	float	`printf ("%f\n", fRealNumber);` `printf ("%f\n", dRealNumber);`	3.000000 339999999999999885680602134547995295744 0.000000

	lf	long float (double)	double	`fRealNumber = dRealNumber;`
				`printf ("%lf\n", fRealNumber);`
				`printf ("%lf\n", dRealNumber);`
					inf
					339999999999999885680602134547995295744 0.000000

CONCLUSIONS

- *Remember that* `signed` *and* `unsigned integers` *are represented differently in computer memory. Using the wrong* **format specifier** *can lead to incorrect results (e.g., using* `%d` *for an* `unsigned int`*).*
- *A* `char` *is essentially a one-byte* `integer`*. Therefore, you can treat a* `char` *variable as either an* ***ASCII character*** *or a small* `integer`*.*
- *The* `float` *data type has a smaller range than* `double`*. Assigning a* `double` *value to a* `float` *variable can result in data loss if the value is too large or too small to be represented accurately*
- *The* `lf` *specifier is used for* `double` *values, as* `d` *is already reserved for decimal(* `integer` *).*

> It's important to remember this distinction.

Format = %[flags][width][.precision][modifier]<type>

- Optional:
  - `Flags` - modifiers that alter the formatting or scanning behavior.
  - `Width` - specifies the minimum width of the output field
    or the maximum number of characters to be scanned.
  - `Precision` - controls the precision of floating-point numbers
    or the maximum number of characters to be scanned for strings.
  - `Modifier` - indicates the data type size (e.g., long, short).
- Required:
  - `Type` - specifies the data type of the variable to be formatted or scanned.

[Width]

Width

```
int x = 5, y = -6; int * z; float f = 3.1234f;
```



Data	Type		Description	Example	Result
Numbers	integer	d	The width value for number (text) representation reserves at least as many characters as are needed to represent that number(text) in ASCII characters (digits). If the width is smaller than the number's (text's) representation, the entire number (text) will be displayed. If the width is greater than the number's (text's) representation, extra spaces will be added on the left side.	`printf("'%1d'\n", y);` `printf("'%4d'\n", y);`	' - 6' ' - 6'
		u			
		c		`printf("'%c'\n", letter);` `printf("'%1c'\n", letter);`	' a'
	floating-point number	f		`printf("'%3f'\n", fRealNumber);` `printf("'%9f'\n", fRealNumber);`	' 3.123400 ' ' 3.123400 '
		lf			
Text	string	s		`printf("'%3s'\n", text);` `printf("'%10s'\n", text);`	' Some Text ' ' Some Text '

Format = %[flags][width][.precision][modifier]<type>

- Optional:
  - `Flags` - modifiers that alter the formatting or scanning behavior.
  - `Width` - specifies the minimum width of the output field
    or the maximum number of characters to be scanned.
  - `Precision` - controls the precision of floating-point numbers
    or the maximum number of characters to be scanned for strings.
  - `Modifier` - indicates the data type size (e.g., long, short).
- Required:
  - `Type` - specifies the data type of the variable to be formatted or scanned.

[precision]

Precision

```
int x = 5, y = -6; int * z; float f = 3.1234f;
```



Data	Type		Description	Example	Result
Numbers	integer	d	precision works the same as width — it reserves a minimum field size. If the precision is greater than the ASCII character representation of the number, leading zeros are added to the left. Precision does not truncate the number! It always displays the full value.	`printf ("'%.1d'\n", x);` `printf ("'%.5d'\n", x);`	' 65' ' 00065'
		u			
		c			

	floating-point number	f	precision determines the number of digits after the decimal point that are displayed. If the number has more decimal places than specified by the precision, it will be truncated.	`printf ("'%.6f'\n", fRealNumber);`
				`printf ("'%.2f'\n", fRealNumber);`
				`printf ("'%.0f'\n", fRealNumber);`
					' 3.123400 '
					' 3.12 '
					' 3'

		lf			

Text	string	s	precision determines the precise number of characters to be extracted from the string. Any characters beyond the specified precision will be discarded.	`printf ("'%.1s'\n", text );`
				`printf ("'%.5s'\n", text );`
				`printf ("'%.20s'\n", text );`
					' S'
					' Some'
					' Some Text '

Format = %[flags][width][.precision][modifier]<type>

- Optional:
  - `Flags` - modifiers that alter the formatting or scanning behavior.
  - `Width` - specifies the minimum width of the output field
    or the maximum number of characters to be scanned.
  - `Precision` - controls the precision of floating-point numbers
    or the maximum number of characters to be scanned for strings.
  - `Modifier` - indicates the data type size (e.g., long, short).
- Required:
  - `Type` - specifies the data type of the variable to be formatted or scanned.

[Flags]

FLAGS

- `+` : Always display the sign of a number, even if it's positive.
- `-` : Left-justify the output within the given field width.
- `0` : Pad the field with zeros instead of spaces.
- `#` : Use an alternative form for the conversion specifier.

Flag +

```
int x = 5, y = -6; int * z; float f = 3.1234f;
```



Data	Type		Description	Example	Result
Numbers	integer	d	Always display the sign of a number, even if it's positive.	`printf ("'%+d\n", x);` `printf ("'%+d\n", y);`	' +65' ' - 6'
		u			
		c			
	floating -point number	f		`printf ("'%+f\n", fRealNumber);`	' +3.123400'
		lf			
Text	string	s	None	None	None

Flag -

```
int x = 5, y = -6; int * z; float f = 3.1234f;
```



Data	Type		Description	Example	Result

Numbers	integer	d	Left-justify the output within the given field width.	`printf("'%7d'", x);`
				`printf("_next\n");`
				`printf("'%-7d'", x);`
				`printf("_next\n");`
					' 65_next'
					'65 _next'

		u			
		c			

	floating-point number	f		`printf("'%13f'", fRealNumber);`
				`printf("_next\n");`
				`printf("'%-13f'", fRealNumber);`
				`printf("_next\n");`
					' 3.123400_next'
					'3.123400 _next'

		lf			

Text	string	s		`printf("'%20s'", text);`
				`printf("_next\n");`
				`printf("'%-20s'", text);`
				`printf("_next\n");`
					' Some Text_next'
					'Some Text _next'

Flag 0

```
int x = 5, y = -6; int * z; float f = 3.1234f;
```



Data	Type		Description	Example	Result
Numbers	integer	d	Pad the field with zeros instead of spaces.	`printf ("'%07d\n", x);`	' 0000065_next '
		u			
		c			
	floating-point number	f		`printf ("'%013f\n", fRealNumber);`	' 000003.123400_next '
		lf			
Text	string	s		`printf ("'%020s", text );`	' 00000000000Some Text '

FLAG#

> This will be explained in the future, because...
>
> To understand recursion, you must first understand recursion.

Format = %[flags][width][.precision][modifier]<type>

- Optional:
  - `Flags` - modifiers that alter the formatting or scanning behavior.
  - `Width` - specifies the minimum width of the output field
    or the maximum number of characters to be scanned.
  - `Precision` - controls the precision of floating-point numbers
    or the maximum number of characters to be scanned for strings.
  - `Modifier` - indicates the data type size (e.g., long, short).
- Required:
  - `Type` - specifies the data type of the variable to be formatted or scanned.

[Modifier]

Modifier

```
short int x = 65;  int y = -69000; float fRealNumber = 3.1234f; double dRealNumber = 3.4e50;
```



Data	Type		Description	Example	Result
Numbers	integer	d	Modifier, h and l specify how many bytes should be formatted as a variable. Therefore, if we use short (h - 1 bit) on a normal int (long - 32 bit), we will get an incorrect result because printf will take only 16 bits and build a number representation from it.	printf ("' %0hd' \n", x); printf ("' %0hd' \n", y); printf ("' %0ld' \n", x); printf ("' %0ld' \n", y);	65 - 3464 65 - 69000
		u			
		c			
	floating -point number	f	Since there's no such thing as hf (as hf is simply f), the compiler ignores h, and l represents a double. As you can see, there's no lf type, only f with the l modifier.	printf ("' %0hf' \n", fRealNumber); printf ("' %0lf' \n", fRealNumber); printf ("' %0hf' \n", dRealNumber); printf ("' %0lf' \n", dRealNumber);	' 3.123400 ' ' 3.123400 ' ' 33999999... ' ' 339999999999999844028425 1433794782958910267457536.000000'
		lf			
Text	string	s	None	None	None

THANK

You